

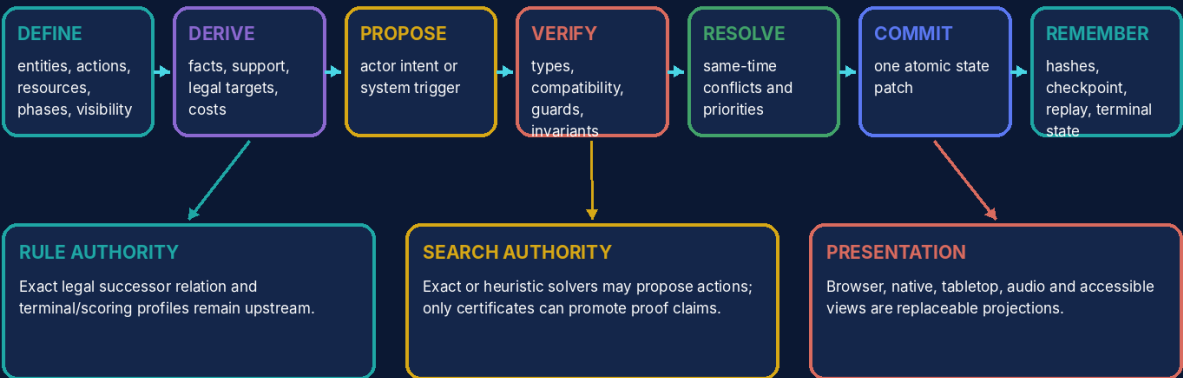
UNIVERSAL GEOMETRIC TOPOLOGICAL
OPEN MODULAR SUBSTRATE

UGTOMS RULEFORGE 1.0

Typed Rule-World Runtime, Deterministic Commit and Replay
Cartridge

UGTOMS RULEFORGE 1.0 architecture

A canonical rule-world cartridge drives deterministic play, solving and teaching adapters.



A rule file is not a world change; verified deterministic commit remains the authority.

Pure domain evaluation may propose evidence; only the retained authority chain may commit state.

ugtoms.runtime.ruleforge@1.0.0

Component	RULEFORGE
Canonical identity	ugtoms.runtime.ruleforge@1.0.0
Component type	Reusable runtime / domain-cartridge host
Version / date	1.0.0 / 30 August 2026
Document type	Open modular cartridge specification and implementation-ready profile
Status	Formal runtime cartridge; specification self-checked; production networking, fairness and game completeness not claimed

ATTRIBUTION AND EVIDENCE BOUNDARY

Prepared for Tom Klootwijk. Requester attribution is recorded as supplied and is not independently verified. This is a technical design artifact, not legal proof, scientific validation, regulatory approval or production safety certification.

Contents and Reading Rule

Section	Purpose
1	Executive decision and release scope
2	Source basis and evidence discipline
3	Open modular cartridge ABI
4	Formal rule-world state
5	Definitions, entities and state cells
6	Actions, support, compatibility and guards
7	Order, simultaneous events and atomic commit
8	Visibility, chance and information state
9	Terminality, scoring and solver boundary
10	Deterministic demonstration
11	Cartridge format, queries and adapters
12	Validation, boundaries and kill criteria
13	Mechanism catalog
14	Claims ledger
15	Applications and final definition

READING RULE

The UGTOMS source line supplies the reusable authority, typing, order, evidence and packing discipline. The specialist domain model in this cartridge is an engineering-derived extension. Where the source corpus is silent, this report says so instead of presenting general domain knowledge as source-derived.

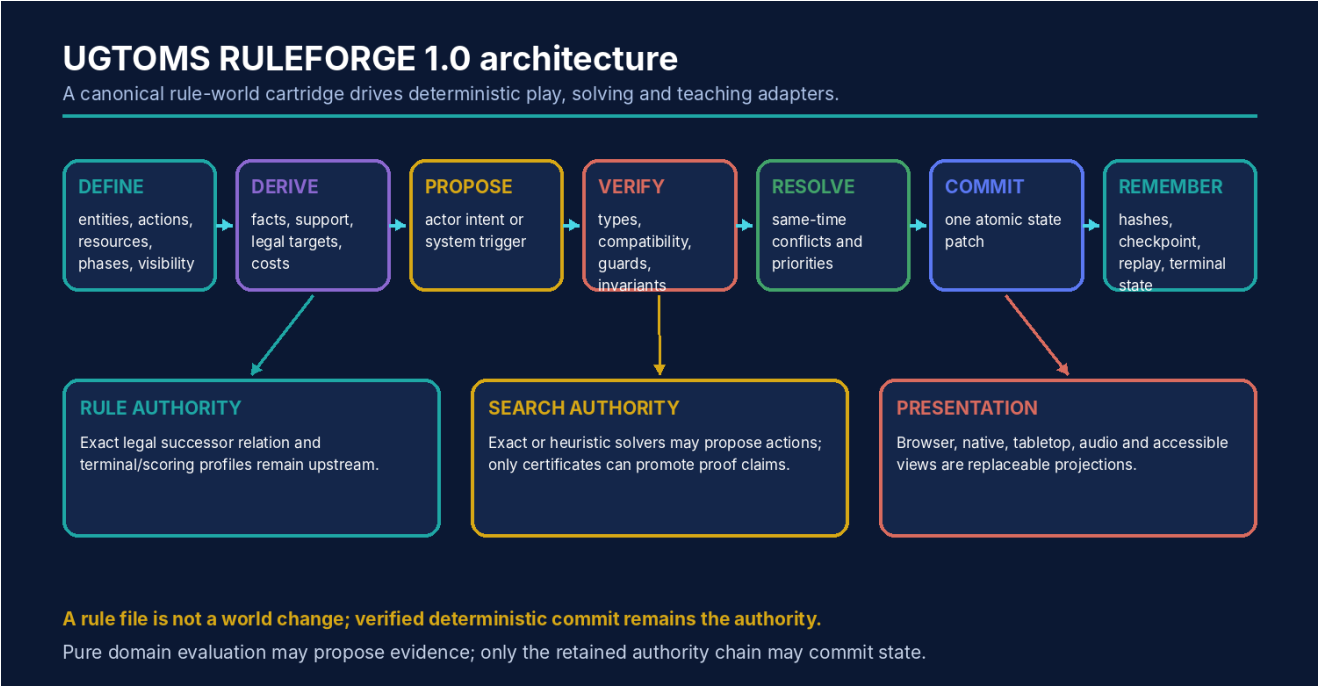
Executive Decision and Release Scope

FORMAL DECISION

GO for RULEFORGE as the first Batch 02 cartridge and the reusable host for data-defined rule worlds. Legal state transitions, timing, visibility, conflicts, scoring and terminality become typed records instead of hidden engine code, while rendering, physics, networking and search remain adapters.

RULEFORGE generalizes the existing Definition Breaker, Chess and Go application patterns without claiming that one schema erases every specialist engine. Its canonical object is the rule world and its exact successor relation; user input, AI, scripts and network messages merely propose actions.

- A cartridge can define board games, card games, turn tactics, puzzle worlds, bounded simulations and educational microworlds.
- The same cartridge feeds validation, deterministic simulation, replay, exact/heuristic search adapters and self-contained presentation.
- Rules, views and search scores remain separate authority classes: presentation never legalizes an action, and search never rewrites the rules.



Source Basis and Evidence Discipline

The cartridge inherits the following UGTOMS commitments:

- Typed definitions and operators live inside the substrate, with stable IDs, dependencies and content hashes [S1-S3].
- Pure evaluation is upstream of support, compatibility, guard, verified proposal, deterministic commit and lineage [S1, S2, S4].
- Coordinates, pictures, sounds, meshes and packed codes are not identity or authority by themselves [S4-S8].
- Uncertainty, exactness, capability and error margins are explicit and may only be strengthened by evidence [S1, S2, S8].
- Component-scoped identities prevent a specialist domain pack from masquerading as a universal replacement of every other release [S10].

ENGINEERING-DERIVED DOMAIN CONTENT

The generic rule-world type system, action records, hidden-information profiles, simultaneous conflict policies, chance profiles and examples are engineering-derived. The uploaded sources provide the authority, definition, order, replay and proof boundaries, not a universal pre-existing game-rule language.

ID	Registered source	Use in this cartridge	Pages
S1	UGTS__FOUNDATION__UNICODE_v5.pdf	Foundation 5.0 OperatorCells, literal/glyph/semantic separation, converse relations, exactness/capability and packed execution boundaries.	pp. 1-21
S2	UGTS_KC_4_2_General_Operator_Order_Addendum(1).pdf	Typed OperatorSpec, dependency and phase order, deterministic reduction, certificates, trace queries and open extension contracts.	pp. 2-32
S3	UGTS_KC_3_6_Tom_Klootwijk.pdf	Definitions as first-class content-addressed records, resolvable references, explicit operation order and deterministic trace.	pp. 1-15
S4	Unified_Geometric_Topological_Substrate_GPU_Native_Addendum.pdf	Canonical support → compatibility → guard → event → transition → lineage authority and narrow packed-state semantics.	pp. 2-15
S5	UGTS_KC_2_0_Tom_Klootwijk_Expanded_Substrate.pdf	Typed state, patterns, topology, bounded dynamics, hybrid events and numerical/error contracts.	pp. 3-15
S6	UGTS_KC_Two_Hands_3_0_Interactive_Graphics_Runtime.pdf	Stable identity, proposal verification, deterministic commit, checkpoints, rollback and replay.	pp. 3-16
S7	UGTS_KC_3_9_KC_Elizabeth_Vector_Game_Runtime.pdf	Versioned game projects, fixed-step runtime, snapshots/hashes, CLI and self-contained HTML delivery.	pp. 2-16
S8	UGTS_KC_4_0_Spatial_Evidence_Ledger(1).pdf	Observation versus authority, uncertainty, provenance, atomic patches, checkpoints and evidence-bearing lineage.	pp. 3-14
S9	UGTS_KC_3_6_2_SCLP_Tom_Klootwijk.pdf	Finite packing, interval guards, winding, bounded grammars and explicit topology/rotation distinctions.	pp. 1-15
S10	UGTS_Versioning_Charter_Phase_1_Foundation_Chronology(1).pdf	Component-scoped identities and explicit release graphs instead of one misleading global version ladder.	pp. 1-6
S11	UGTS_KC_Definition_Breaker_DB_0_1_Concept(1).pdf	Rules as typed physical objects, candidate preview, deterministic verification, commit, replay and reason-coded rejection.	pp. 2-24
S12	UGTS_KC_CHESS_1_0_Proof_Carrying_Solver(1).pdf	Exact rule authority, bounded proof certificates, heuristic separation and independently verifiable replay.	pp. 2-20
S13	UGTS_KC_4_2_GO_Solver_Report(1).pdf	History-correct exact state, finite minimax certificates, search/non-proof separation and self-contained delivery.	pp. 1-15

Open Modular Cartridge ABI

The cartridge is a concrete instance of the shared domain object D_RULEFORGE:

D = (T, O, R, U, I, Q, E, G, Delta, L, Cert, K, Pi)	
Field	Contract
T	Entity, value and state types
O	Typed operators and transformations
R	Relations, graphs, topology and converse pairs
U	Units, dimensions, coordinate and profile conventions
I	Invariants, conservation laws and constraints
Q	Dependency, precedence, phase and reduction order
E	Observations, evidence, uncertainty and provenance
G	Guards, thresholds and admission predicates
Delta	Proposed and committed state patches
L	Lineage, replay, history and migration
Cert	Exactness, residual, interval, scope and status certificates
K	Cold atlas, hot codebook and packed representation
Pi	Optional visual, audio, physical or device projection

A conforming cartridge contains a manifest, type registry, operator registry, relation atlas, unit/profile system, dependency DAG, guard library, transition definitions, claims ledger, examples and validation fixtures. External mature libraries may sit behind adapters when they are safer or more accurate than a fresh reimplementation.

General Operator Record and Order

```
OperatorSpec = (id, version, syntax, arity, domains, codomain, units, shape,
               laws, exactness, effects, capabilities, dependencies,
               order_contract, invariants, provenance, content_hash)
```

The record follows the General Operator and Order layer [S2]. A symbol or glyph is not allowed to smuggle in undeclared type, unit, order, inverse or authority semantics.

EXAMPLE RECORD

ruleforge.action.apply consumes a complete serialized state plus an ActionProposal and returns either a reason-coded rejection or an immutable StatePatch. It cannot mutate state, enumerate hidden information outside policy, or invoke scoring/terminality before the patch has passed deterministic conflict resolution.

Normative phase schedule

Phase	Meaning
1-3	parse, normalize and resolve content-addressed references
4-6	type, shape and unit validation; dependency closure
7-9	construct candidates; apply declared composition and reduction order
10-12	evaluate exact or bounded results; issue capability and residual certificate

Phase	Meaning
13	support, compatibility and guard classification
14	verified proposal and deterministic conflict resolution
15	atomic commit, lineage, replay and optional projection

One cartridge, several execution targets

Rules remain one canonical record; solvers, editors and presentations are downstream adapters.

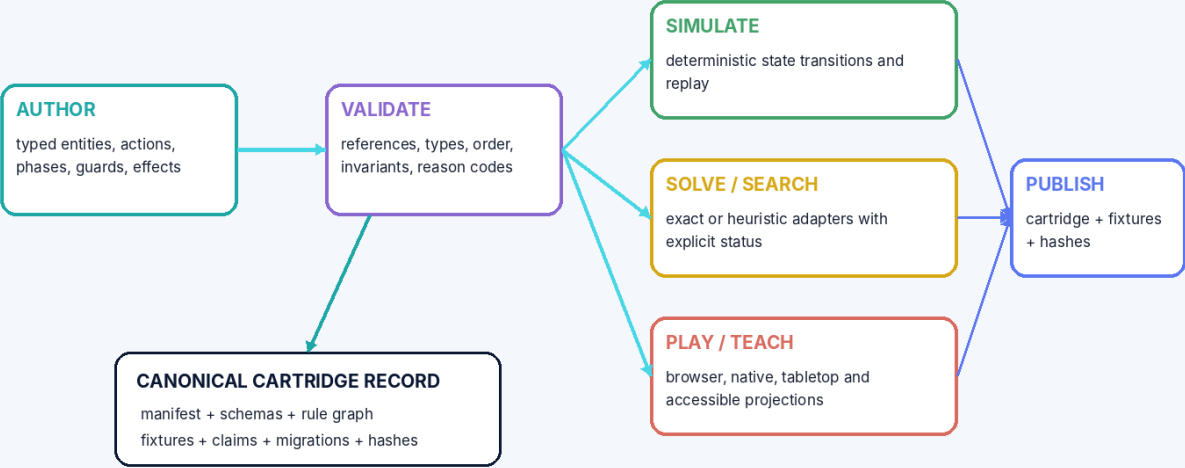


Figure 2. One canonical cartridge record can support several downstream execution targets.

Formal Rule-World State

q_RF = (D, X, A, R, P, V, C, H, L, U)		
D definitions/profile X entity and relation state A actors/roles/authority R resources and counters P phase, priority and time V visibility/knowledge partitions C chance source state H repetition/history context L event/checkpoint lineage U uncertainty/status		
Object	Authority role	Boundary
RuleWorldProfile	Turn/time model, visibility, chance, conflict, terminal and migration semantics.	A project title or renderer cannot override the profile.
EntityType / Entity	Stable definition and instance identity with typed state fields.	Coordinates, sprites and labels are representations, not identity alone.
ActionSpec	Typed actor/target parameters, costs, guards, effects and failure reasons.	Input devices and AI only produce proposals.
StatePatch	Finite immutable changes and derived invalidation set.	No partial write before the complete patch is verified.
TerminalCertificate	Win/loss/draw/finish status plus score and proof scope.	A heuristic evaluation is not a terminal certificate.
IDENTITY AND INFORMATION SPLIT		
world state != actor knowledge != public projection != replay history. A hidden card can exist in authoritative state while remaining absent from one actor projection; learning the card is a separate information event.		

Definitions, Entities and State Cells

```
RuleWorldDefinition = (manifest, types, entities, relations, resources, actions,
    phase_schedule, visibility, chance, invariants,
    terminal_rules, scoring_rules, migrations, fixtures, hash)
```

State-cell capability classes

Class	Example	Required contract
Finite enum	door = closed/open/locked	Complete value set and transition policy.
Integer/resource	energy = 0..10	Range, overflow and conservation/cost policy.
Set or multiset	inventory, hand, captured pieces	Element identity, cardinality and order semantics.
Graph/topology	rooms, board adjacency, routes	Stable nodes/edges and explicit gluing/neighbor rules.
Clock/timer	turn timer, cooldown, continuous hazard	Time unit, update source, pause and threshold policy.
Uncertain/observed	sensor-backed educational or mixed reality fact	Evidence, uncertainty and promotion gate.
SCHEMA RULE		
Every state field names its type, default/initial policy, authority class, serialization, visibility and legal writers. Hidden engine variables are rejected unless intentionally declared as implementation-only caches with no semantic authority.		

Derived facts

Pure derived facts such as attack maps, reachability, legal-target sets or resource totals use dependency-aware cache records. They may accelerate queries but never become independent state authority; invalidation follows the committed patch.

Actions, Support, Compatibility and Guards

```
ActionSpec = (id, actor_roles, parameters, target_query, costs, support,
               compatibility, guards, invariants, effect_builder,
               priority, interrupt_policy, reason_codes, content_hash)
```

Stage	Typical questions	Failure result
Support	Is the actor in range, in the correct phase, route, zone or scope?	out-of-support
Compatibility	May this role, object type, owner/team and rule profile interact?	type/role/ownership mismatch
Cost/precondition	Are resources, cooldowns, permissions and dependencies present?	missing-resource or dependency
Guard	Has the required boundary, occupancy, time, history or trigger occurred?	guard-false or ambiguous
Invariant	Would the complete patch preserve declared world promises?	invariant-violation
Commit	Did conflict resolution select this proposal and do preconditions still match pre-state?	stale-prestate or conflict-lost
FIRST-FAILURE EXPLANATION		
The verifier returns the first stable reason code under the declared order, plus a full trace on request. Rejection is deterministic evidence, not a generic invalid-action animation.		

Persistent versus discrete rules

- Persistent constraints remain checked while their condition holds; they are not repeated event spam.
- Discrete triggers fire at a declared transition or edge and record exactly one event per policy.
- Continuous or physics-backed systems may propose bounded events through adapters, but commit semantics remain discrete and replayable.

Order, Simultaneous Events and Atomic Commit

Order	Contract
Definition/dependency	Resolve types, actions and derived facts before evaluating proposals.
Phase	Upkeep, input, reaction, resolution, transition, terminal and lineage phases are explicit.
Same-time proposal	Collect a complete batch from the same authoritative pre-state.
Conflict	Use stable priority, initiative, resource locking, merge or reject rules with reasons.
Atomic patch	Apply the selected patch set once; no proposal observes another half-applied patch.
Terminal/scoring	Evaluate after commit under a declared schedule; score cannot retroactively legalize the move.
<pre>resolve(batch, q_pre) -> (accepted_patches, rejected_reasons, trace) q_post = atomic_apply(q_pre, canonical_join(accepted_patches))</pre>	

Example conflict policies

Policy	Use	Boundary
initiative then stable ID	turn tactics and interrupts	initiative source and tie-break are part of state/profile
exclusive resource lock	two actors claim one object	losers receive conflict-lost without partial costs
commutative merge	independent counters or set additions	only when the declared patch algebra proves compatibility
all-or-none transaction	cooperative simultaneous action	every member passes or the group patch rejects

Visibility, Chance and Information State

Object	Semantics	Non-claim
VisibilityPolicy	Maps authoritative state to actor/team/public projections.	Does not itself secure memory or network traffic.
InformationEvent	Actor learns, reveals, forgets or verifies a proposition about world state.	Knowledge is not inferred merely from proximity.
RandomSourceProfile	Algorithm, seed, domain, sample mapping and consumption order.	A pseudorandom sample is not physical entropy.
ChanceCertificate	Source pre-state, sample, mapping, result and post-state.	Reproducibility does not imply fairness.
chance proposal -> source/support/phase guard -> deterministic sample mapping -> action proposal -> ordinary verification -> commit + lineage		
PARTIAL-OBSERVATION RULE		
Exact solvers and replays consume authoritative state. Player-facing agents consume only their authorized information state unless an omniscient analysis profile is explicitly selected and labeled.		

Accessibility and interaction

Definition Breaker contributes a practical presentation pattern [S11]: relevant typed sockets, safe preview, reason-coded rejection, checkpoint-based undo and multiple input routes. RULEFORGE stores these as projection/access profiles without creating a lower-status rule set.

Terminality, Scoring and Solver Boundary

Layer	Exact authority	Boundary
Rules	Legal successors, costs, captures/effects and terminal checks.	Must be deterministic under the selected profile.
Position proof	Complete finite search/certificate for one serialized state.	Proof scope names exact profile, history and budgets.
Heuristic search	Ranked actions, rollout values or learned estimates.	Scores are proposals/estimates, never legality or proof.
Game/world solution	Certificate for the declared initial state and complete rule profile.	Not claimed merely because bounded fixtures solve.

The Chess and Go packages demonstrate this separation [S12-S13]: exact rule kernels and bounded certificates coexist with heuristic search without allowing the heuristic layer to redefine the game.

```

successors(q) = [ commit(q,a) for a in legal_actions(q) ]
exact solver: closes complete finite graph -> certificate
heuristic adapter: budget exhausted -> estimate + UNKNOWN proof status

```

Deterministic Demonstration

RULEFORGE deterministic action trace

A rejected proposal and a valid sequence use the same typed rule records.

REJECTED: open_door before take_key

support: door nearby

compatibility: actor may use door

guard: key possessed

commit: blocked

ACCEPTED LINEAGE

1 take_key

inventory + key; room - key

2 open_door

door.closed → false

3 move_to_b

actor.location A → B

4 goal rule

terminal: reached

Every step stores proposal ID, first-failure reason or patch, pre/post hash and replay label.

Figure 3. The same action records produce a stable rejection and a replayable accepted sequence.

Proposal	Result	Reason / patch
open_door(actor, door)	REJECT	guard key-possessed = false
take_key(actor, key)	ACCEPT	inventory + key; room contents - key
open_door(actor, door)	ACCEPT	door.closed true → false
move(actor, room_b)	ACCEPT	actor.location room_a → room_b
terminal goal-reached	TRUE	finish certificate after fourth commit
two actors claim one gem	ONE ACCEPT	exclusive-resource conflict resolved by declared initiative/tie-break

Canonical queries

- legal_actions(actor_id, state_hash) → ordered actions plus optional first-failure reasons
- preview(action) → candidate patch, affected derived facts, conflicts and projected consequence
- trace(event_id) → definitions, guards, pre/post values, hashes and terminal/scoring effects
- view(actor_id) → authorized projection plus information lineage
- replay(checkpoint, events) → reconstructed state and divergence report

Cartridge Format, Packing and Adapters

Package layer	Required content
Manifest	component identity, schema, parent capabilities, migration, hashes and non-claims
Definitions	types, entities, actions, relations, resources, phases, visibility, chance and rules
Validation	schema checks, dependency closure, trace fixtures, hash/replay and conflict cases
Runtime ABI	state/proposal/patch/certificate records plus deterministic query interface
Adapters	input, UI, audio, accessibility, physics, networking, solver and storage boundaries
Publication	self-contained project or signed bundle with source map and checksums

PACKING RULE

Hot IDs, opcodes and bitfields are only aliases into the exact cold cartridge. Unknown codes reject. Seed reconstruction is allowed only for closed deterministic content; player input, network events and external observations remain novelty records.

Adapter	Allowed authority
Renderer/audio/accessibility	Projection only; may request actions but not commit state.
Physics/model	May propose collision/contact/timing events with error/status.
Network	Transports proposals, accepted events and checkpoints; server/peer authority is profile-bound.
Search/AI	Ranks or proposes legal/near-legal actions; exact proof requires certificate.
Editor	Creates/migrates definitions and candidate worlds; published cartridge passes conformance checks.

Reference Validation, Boundaries and Kill Criteria

REFERENCE SELF-CHECK RESULT

28/28 deterministic specification self-checks passed. These checks cover catalog uniqueness, required fields, dependency closure, demo arithmetic and declared reason codes. They are package evidence, not independent domain validation.

Checked area	Result
Catalog integrity	48 unique RFG IDs; required fields present.
Rule-world fixture	Key-door-goal sequence produces one stable rejection and four accepted lineage events.
Atomicity	Conflict fixture spends no cost on the rejected claimant.
Replay	Accepted event sequence reconstructs the same canonical post-state hash.
Solver boundary	Heuristic adapter fixture cannot emit an exact certificate.
Visibility boundary	Actor projection excludes unauthorized private fields while preserving public event reasons.

Explicit non-claims

- No universal game engine, production networking stack, anti-cheat system or fairness certification is claimed.
- No existing game is represented unless a complete cartridge and validation suite are supplied.
- No heuristic, learned model or rollout score is promoted to exact proof.
- No hidden-information security is proven by a view filter alone.
- No performance or compression advantage is claimed without equal-semantics measurement.

Kill criteria

1. Reject or narrow a use when a symbol, codebook slot or packed record cannot resolve to its exact global definition.
2. Reject when a capability or exactness label is stronger than the model, measurement or external source supports.
3. Reject when units, profiles, coordinate systems, operand order or dependency order are implicit.
4. Reject when a proposal bypasses support, compatibility, guards, deterministic commit or lineage.
5. Reject when an external adapter is required for safety or conformance but its version, calibration or provenance is absent.
6. Reject any performance, compression or safety claim without a named workload, target, error/equivalence contract and evidence.

Complete RULEFORGE 1.0 Mechanism Catalog

The catalog contains 48 cartridge-local mechanisms. These IDs are component-scoped and do not silently extend a global M-number ladder.

ID	Family	Mechanism	Core contract
RFG001	Package	Cartridge manifest	Canonical component identity, schema versions, dependencies, capabilities and hashes.
RFG002	Package	Rule-world profile	Declares turn model, time model, visibility, conflict policy, terminality and migration profile.
RFG003	Identity	Entity type	Stable typed definition for a class of world objects and allowed state fields.
RFG004	Identity	Entity instance	Stable instance ID, type reference, state and lineage; coordinates are not identity.
RFG005	Identity	Actor or player identity	Role, authority, team and control channel remain explicit and versioned.
RFG006	Identity	Resource type	Typed countable, continuous or unique resource with unit and conservation policy.
RFG007	State	Typed state cell	Finite scalar, enum, set, relation or structured value with validation contract.
RFG008	State	Support region	Declares where an action or rule may matter in graph, space, phase or scope.
RFG009	State	Ownership and role state	Tracks who controls, possesses, targets or may mutate an entity.
RFG010	State	Visibility partition	Separates public, private, team and unknown state under an explicit policy.
RFG011	State	Turn and phase state	Current actor, round, phase, subphase and priority are authoritative fields.
RFG012	State	Clock and timer state	Discrete or bounded continuous time with pause, expiration and ordering semantics.
RFG013	Action	ActionSpec	Typed parameters, actor role, target selectors, costs, guards, effects and reason codes.
RFG014	Action	Parameter binding	Binds literal input to typed action ports after validation and coercion policy.
RFG015	Action	Target selector	Finite query returning candidate entities, cells, routes or relations in support.
RFG016	Action	Cost and precondition	Checks resources, permissions, cooldowns and other non-spatial requirements.
RFG017	Action	Effect patch	Immutable bounded additions, removals and replacements proposed for authoritative state.
RFG018	Action	Cancel and interrupt policy	Declares when a pending action may be cancelled, interrupted or resumed.
RFG019	Rule	Compatibility relation	Determines whether actor, target, resource and current profile may couple.
RFG020	Rule	Guard predicate	Classifies readiness from thresholds, occupancy, history, time or relation state.
RFG021	Rule	Invariant	Property that every accepted patch must preserve or explicitly transition.
RFG022	Rule	Terminal rule	Declares win, loss, draw, finish, abort or unresolved terminal state.
RFG023	Rule	Scoring rule	Typed evaluation downstream of legal state; score never overrides legality.
RFG024	Rule	Derived-fact rule	Pure inference over current state with dependency and cache invalidation contract.

ID	Family	Mechanism	Core contract
RFG025	Order	Dependency DAG	Definitions and derived facts resolve before actions that consume them.
RFG026	Order	Phase schedule	Explicit parse, derive, propose, verify, commit, terminal and lineage order.
RFG027	Order	Simultaneous proposal batch	Collects same-time intents before deterministic resolution.
RFG028	Order	Conflict resolution	Stable priority, initiative, merge or rejection policy with reason codes.
RFG029	Order	Deterministic reduction	Canonical order for sums, votes, areas, scores or aggregated effects.
RFG030	Order	Event priority	Orders interrupts, reactions, triggers and terminal checks without hidden scheduler state.
RFG031	Information	Public-state projection	Loss-reporting view of authoritative state available to all actors.
RFG032	Information	Private-state projection	Actor-scoped view with noninterference and authorization contract.
RFG033	Information	Information event	Records reveal, learn, forget or observe without conflating knowledge and world state.
RFG034	Information	Reveal and conceal patch	Changes visibility/knowledge state through a verified event.
RFG035	Chance	Random-source profile	Seed, algorithm, sample domain and consumption order are explicit.
RFG036	Chance	Chance-resolution certificate	Records source state, sample, mapping and resulting proposal; randomness is not authority.
RFG037	Authority	Proposal record	Actor, action, parameters, support, priority, pre-hash and requested effects.
RFG038	Authority	Verification trace	First-failure or full-pass trace across types, support, compatibility, guards and invariants.
RFG039	Authority	Atomic commit	Applies one conflict-resolved patch or rejects without partial mutation.
RFG040	Authority	Pre/post hash	Canonical state identity before and after commit with migration profile.
RFG041	Authority	Checkpoint	Snapshot and lineage position for replay, return, undo or network resynchronization.
RFG042	Authority	Replay and migration	Replays events under their original schema or an explicit migration adapter.
RFG043	Query	Legal-action enumeration	Returns finite supported actions with optional reason/explanation queries.
RFG044	Solver	Exact solver adapter	Consumes the legal successor relation and emits proof only when a finite search closes.
RFG045	Solver	Heuristic/search adapter	Produces ranked proposals or estimates with budgets and explicit non-proof status.
RFG046	Validation	Scenario fixture	Canonical initial state, scripted proposals, expected reasons, hashes and terminal result.
RFG047	Delivery	Projection adapter	Browser, native, tabletop, audio or accessible view cannot bypass authority.
RFG048	Governance	Cartridge conformance report	Validates schemas, IDs, dependencies, traces, fixtures, hashes, limits and non-claims.

Claims Ledger

Claim	Disposition	Reason
One cartridge can drive multiple runtimes.	ADMIT / bounded	Valid when all targets consume the same canonical definitions and preserve event semantics.
A rule graph replaces all game engines.	REJECT	Rendering, networking, physics, audio and domain-specific solvers remain adapters or separate systems.
Legal move generation is exact.	ADMIT / profile	Exact relative to a complete finite rule-world profile and serialized state.
Heuristic search proves the result.	REJECT	Only a complete exact search and independently checked certificate may prove a bounded result.
Seeded chance is reproducible.	ADMIT / bounded	Reproducible only with exact algorithm, seed, consumption order and mapping profile.
Replay is undo.	CORRECT	Replay can reconstruct earlier states; a legal undo remains a domain rule or editor operation.
Same coordinate means same object.	REJECT	Stable identity, ownership, sheet, phase and lineage remain separate.
A score can override legality.	REJECT	Scoring is downstream of authoritative legal transitions.
Hidden state is secure because it is omitted from the view.	REJECT	Security requires an implementation and threat model; a projection contract alone is not secrecy proof.
A deterministic runtime is automatically fair.	REJECT	Fairness depends on rules, information, accessibility and governance, not determinism alone.
Compact opcodes are self-defining.	REJECT	They require a versioned global codebook and exact definition hashes.
RULEFORGE is production complete.	REJECT	This release freezes a cartridge/runtime contract and reference fixtures, not a commercial engine.

High-Impact Application Profiles

Profile	What the cartridge enables
Universal board/card/tactics host	Load multiple deterministic rule worlds from cartridges instead of separate hardcoded legality engines.
Definition Breaker authoring runtime	Turn typed rule objects, preview, reason codes and commit benches into a reusable systemic-game foundation.
Proof-carrying game analysis	Attach exact bounded solvers or heuristic search without letting scores rewrite the rules.
Education and physical play	Render the same rule object as screen, audio, tactile, printable or tabletop interactions.
Protocol and workflow simulator	Reuse actions, phases, conflicts and lineage for bounded non-game rule systems.

Final Formal Definition

FORMAL DEFINITION

UGTOMS RULEFORGE 1.0 is a component-scoped reusable rule-world runtime in which entity identity, typed state, actions, visibility, chance, phase/order, conflicts, terminality and scoring are literal cartridge data; every action remains a proposal until support, compatibility, guards, invariants and deterministic conflict resolution accept one atomic patch; and presentation, networking, physics and search remain scoped adapters with replayable lineage.

Implementation sequence

- Freeze schemas for RuleWorldProfile, EntityType, ActionSpec, StatePatch and TerminalCertificate.
- Implement the dependency resolver, verifier, conflict resolver, canonical serializer and replay oracle.
- Ship the key-door-goal and simultaneous-resource fixtures as the first conformance suite.
- Build a self-contained browser editor/player with accessible safe preview and trace queries.
- Add optional Chess, Go and Definition Breaker adapters while preserving their specialist rule profiles.

RELEASE CONCLUSION

This 1.0 document freezes a domain-facing contract. It is deliberately useful before a production engine exists: implementations can now disagree visibly through typed profiles, reason codes and certificates instead of hiding differences behind the same symbol.